

Лабораторная работа № 6

Тема: Управление процессами, фоновыми заданиями, перенаправлением потоков и сборкой конвейеров (pipeline) в Linux.

Цель работы: Закрепить практические навыки работы с процессами и потоками данных в Linux. Научиться проводить комплексную диагностику системы в реальном времени (htop) и через моментальные снимки (ps), управлять жизненным циклом задач (перевод в фон, переключение между foreground/background), использовать перенаправление дескрипторов (stdin, stdout, stderr) и строить сложные конвейеры фильтрации с помощью утилит grep, cut, sort, uniq и wc.

Необходимое ПО

- Эмулятор терминала для подключения по SSH.
- Развернутый Linux-сервер (Ubuntu/Debian).

Ситуация

Вы работаете системным администратором. Нагрузка на один из серверов компании резко возросла, и вам необходимо оперативно локализовать проблему. Задачи, которые перед вами стоят:

1. Выявить в реальном времени «зависший» процесс, потребляющий ресурсы, изучить его происхождение (родительский PPID) и принудительно остановить с помощью сигналов.
2. Продемонстрировать навыки управления длительными задачами, запустив фоновый процесс генерации отчетов, не блокируя терминал.
3. Собрать конвейер (pipeline) обработки системных конфигураций, чтобы без использования ручного поиска вычленили статистику по используемым в системе командным оболочкам.

Требования к выполнению работы

1. **Контроль сигналов:** Принудительное уничтожение процессов сигналом SIGKILL (-9) применять только в крайнем случае, если процесс полностью игнорирует штатный запрос на завершение SIGTERM (-15).
2. **Точность конвейеров:** При сборке конвейеров утилита uniq должна применяться **строго после** предварительной сортировки данных (sort), иначе результат подсчета будет неверным.

3. **Изоляция ошибок:** При перенаправлении вывода четко разделяйте стандартный поток stdout (>) и поток ошибок stderr (2>).

Порядок выполнения работы

Этап 1. Интерактивный аудит и поиск процессов (htop / ps)

1. Подключитесь к серверу по SSH под своей рабочей учетной записью.
2. Откройте интерактивный диспетчер процессов htop (если не установлен, используйте top).
3. Изучите общие показатели системы: загрузку CPU, RAM и показатели Load Average.
4. Переключите отображение в режим **дерева процессов (Tree View)**, нажав клавишу **F5**. Найдите вашу текущую сессию sshd, дочерний процесс оболочки bash и запущенный внутри него htop. Зафиксируйте иерархию.
5. Выйдите из htop (F10 или q).
6. Выполните точечный снимок процессов для вывода PID, PPID, владельца и состояния одной командой:

```
ps -eo pid,ppid,user,stat,comm
```

Найдите в списке процесс со статусом Ss. В отчете поясните, что означает этот статус применительно к найденной программе.

Этап 2. Управление заданиями в shell (Foreground / Background)

1. Симулируйте длительную фоновую задачу (например, непрерывный вывод пингов), перенаправив вывод в «черную дыру» /dev/null, и запустите её в фоне с помощью символа **&**:

```
ping 8.8.8.8 > /dev/null &
```

2. Проверьте список активных фоновых заданий в текущей сессии с помощью команды:

```
jobs
```

3. Верните запущенную задачу на передний план (в интерактивный режим foreground) с помощью команды:

```
fg %1
```

4. Приостановите выполнение задачи (отправьте её в сон/заморозку), нажав комбинацию клавиш **Ctrl+Z**. Система выдаст статус **Stopped**.
5. Снова отправьте её работать, но уже внутри фонового режима, командой:

```
bg %1
```

6. Убедитесь через команду `jobs`, что статус сменился на **Running**.

Этап 3. Сигналы и принудительное завершение процессов (kill)

1. Узнайте точный идентификатор **PID** запущенного вами процесса `ping`. Для этого используйте конвейер:

```
ps aux | grep ping
```

2. Попробуйте штатно и мягко завершить этот процесс, отправив сигнал **SIGTERM**:

```
sudo kill -15 [PID_процесса_ping]
```

3. Проверьте команду `jobs`. Если процесс всё ещё активен, уничтожьте его жестко сигналом **SIGKILL**:

```
sudo kill -9 [PID_процесса_ping]
```

Этап 4. Перенаправление потоков ввода-вывода (>, >>, 2>)

1. Выполните команду просмотра несуществующего файла, перенаправив поток ошибок в отдельный файл лога:

```
cat nonexistent_file.txt 2> error.log
```

2. Убедитесь, что текст ошибки не вывелся на экран, а записался в файл `error.log` (проверьте через `cat error.log`).
3. Запишите стандартный информационный вывод команды `whoami` в новый файл конфигурации:

```
whoami > current_user.txt
```

Этап 5. Сборка сложных конвейеров обработки текста (pipeline)

Вам необходимо получить аналитику из файла базы пользователей `/etc/passwd`.

1. Напишите однострочный конвейер, который:

- Вырезает только последний столбец (заданную командную оболочку shell) из файла /etc/passwd, используя двоеточие : в качестве разделителя (cut).
- Сортирует полученные строки по алфавиту (sort).
- Схлопывает дубликаты и подсчитывает, сколько раз в системе используется каждая оболочка (uniq -c).
- Сортирует итоговый список по численному значению в обратном порядке — от самых популярных к менее популярным (sort -nr).

Пример итоговой команды:

```
cut -d ':' -f 7 /etc/passwd | sort | uniq -c | sort -nr
```

2. Модифицируйте этот конвейер, добавив в самый конец утилиту wc -l, чтобы посчитать, сколько **всего уникальных видов командных оболочек** прописано на данном сервере.

Отчет должен содержать:

- **Скриншот №1 (Этап 1):** Окно программы htop строго в режиме дерева процессов (F5), где четко видна ветка взаимосвязи sshd -> bash -> htop. В prompt терминала должно быть видно имя вашего хоста.
- **Скриншот №2 (Этап 2):** Вывод команды jobs, подтверждающий, что запущенный в фоне ping находится в состоянии Running после манипуляций с Ctrl+Z и bg.
- **Скриншот №3 (Этап 3-4):** Команда поиска PID процесса ping, отправка сигнала утилитой kill и проверка успешного перенаправления ошибки в файл error.log.
- **Скриншот №4 (Этап 5):** Полный экран терминала с вводом финального конвейера (cut | sort | uniq | sort) и полученным результатом подсчета распределения оболочек.

Контрольные вопросы:

1. В чем принципиальная разница между понятиями «программа» и «процесс» в Linux? Может ли одна программа породить несколько процессов?

2. Что такое PID и PPID? Какому родительскому процессу переподчиняются процессы-сироты, если их оригинальный родитель аварийно завершил работу?
3. Расшифруйте статус процесса Ss. Что означает заглавная буква, и какую техническую роль играет строчная буква?
4. Опишите состояние процесса Z (Zombie). Как они появляются и представляют ли они прямую опасность для оперативной памяти сервера?
5. В чем разница между воздействием на процесс сигналов SIGTERM (15) и SIGKILL (9)? Какой сигнал и почему сисадмин должен применять в первую очередь?
6. Как запустить задачу в фоновом режиме изначально? С помощью каких команд можно вернуть фоновую задачу на передний план терминала или приостановить её?
7. Объясните принцип работы конвейера (pipeline, символ |). Какими дескрипторами потоков обмениваются команды, стоящие слева и справа от этого символа?
8. Почему утилита `uniq` требует, чтобы перед ней в конвейере обязательно стояла команда `sort`? Что произойдет, если нарушить этот порядок при анализе логов?